AIOps 监控指标异常检测项目讲稿

适用场景：课堂 15–30 分钟项目汇报
说明：本讲稿按 `data-pj-pre.pptx` 的 20 页顺序编写。实际讲解时可以根据时间压缩，重点讲第 2–16 页，第 17–18 页做总结，第 19 页讲分工，第 20 页作为附录备用。

第 1 页：封面

各位老师、同学大家好，我们小组汇报的题目是 **AIOps 监控指标异常检测**。

第 2 页：项目总览：基于 Spark 的 AIOps 异常检测流程

我们的目标是：**从四千多万条监控指标中，识别异常点、异常序列和多指标融合异常事件。**

整体流程分成两部分。

上半部分是数据处理流程。老师提供的是 `metric.zip` 原始数据，里面包含 6 个场景和 5 类指标。我们先把原始数据上传到 HDFS，也就是 `/aiops/metric` 目录，用 HDFS 做集中式分布式存储。之后使用 Spark 读取并清洗这些数据，完成类型转换、空值过滤，以及 service 指标的宽表转长表。最后输出统一的 `clean_metrics`，一共包含 44,972,147 条记录，并构造出 8,730 条时间序列。

下半部分是算法迭代路线。我们基于同一份 `clean_metrics` 逐步实现四个版本：

V1 是 IQR Baseline，用来建立无监督检测基线；
V2 是 IQR 与 Z-score 的交集，用来提高异常置信度；
V3 是 Window Z-score，用滑动窗口捕捉局部时间异常；
V4 是 MultiMetric Fusion，用来发现多指标共同异常。

第 3 页：原始数据

原始数据从场景上看，它包含 2 天、3 个 cloudbed 场景，也就是 6 个实验场景。从指标类型上看，每个场景下都有 5 类指标：`service`、`node`、`container`、`jvm` 和 `istio`。数据来源多。

这五类指标对应系统的不同层级：

`service` 是业务服务层指标，比如请求数、成功率、平均响应时间；
`node` 是节点层指标，比如系统负载和 IO；
`container` 是容器层指标，比如 CPU、内存和网络；
`jvm` 是 Java 虚拟机指标，比如 GC、线程、内存池；
`istio` 是服务网格指标，比如请求流量、响应状态和代理相关指标。

其中，请求数、响应时间、内存、CPU、成功率等指标单位和数值范围完全不同。量纲差异大

所以我们不能直接对所有 `value` 使用一个统一阈值，而必须先做数据清洗和分组建模。

第 4 页：Spark 清洗：统一成 clean_metrics

这一页讲 Spark 清洗。

清洗阶段我们主要做了三件事。

第一，统一格式。原始数据里，`service` 是宽表，一行包含 `rr`、`sr`、`mrt`、`count` 多个指标；而 `node`、`container`、`jvm`、`istio` 更接近长表，一行是一个 `kpi_name` 和一个 `value`。因此我们把 service 宽表转换成长表，让所有数据都变成“一行一个指标值”的统一结构。

第二，支持时间序列分析。清洗后的字段包括 `timestamp`、`object_id`、`kpi_name`、`metric_type` 和 `value`。其中：

```
object_id + kpi_name + metric_type
```

可以唯一确定一条指标时间序列。例如 `frontend-http + mrt + service` 就表示 frontend-http 服务的平均响应时间序列。

第三，支持后续算法复用。V1 到 V4 都直接读取这份 `clean_metrics`，避免每个算法重复解析原始 CSV。这样也更符合大数据处理流程：先统一清洗，再复用中间结果。

清洗后共有 44,972,147 条记录，构造出 8,730 条时间序列。

第 5 页：V1 IQR Baseline 算法介绍

数据清洗完成后进入算法部分。第一个版本是 IQR Baseline。

IQR 是四分位距。我们先计算 Q1，也就是 25% 分位数；再计算 Q3，也就是 75% 分位数。IQR 等于 Q3 减 Q1。之后使用：

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

作为正常范围。如果某个 value 小于 lower_bound，或者大于 upper_bound，就判为异常。

从数据处理角度看，V1 不是对所有数据算一个统一阈值，而是先从 `clean_metrics` 中筛选某个 `metric_type`，再按：

```
object_id + kpi_name + metric_type
```

分组。这样每条指标序列都有自己的 IQR 阈值。因为不同对象、不同指标、不同类型的数值范围差异很大。如果用全局阈值，就会把请求数、响应时间、JVM 内存、容器 CPU 这类完全不同量纲的指标错误地放在一起比较。

V1 的输出分成两类：`stats` 存每条序列的统计阈值，比如 Q1、Q3、IQR、上下界；`anomaly_points` 存最终被判为异常的时间点。

V1 的优点是无监督、简单、可解释，作为 baseline是很合适的。 但是它的局限是只看分位数边界，比较敏感，可能把一些普通波动也标成异常。

第 6 页：V1 IQR Baseline 结果分析

这一页是baseline的结果。

V1 在全量 44,972,147 条记录中检测出 574,873 个异常点，整体异常率约为 1.28%。

从不同 `metric_type` 看，container 和 istio 的异常点最多。这一方面和它们本身数据量较大有关，另一方面也说明系统资源层和服务网格层的指标波动比较明显。

从这张图我们能看出：

第一，V1 作为无监督 baseline 是有效的。它不需要人工标签，只根据每条时间序列自身的分位数分布，就能在大规模监控数据上发现异常。

第二，V1 的结果数量较多。574,873 个异常点说明 IQR 比较敏感，因此后续需要进一步提高异常置信度。

V1 在无标签大规模数据上建立异常检测基线，但还没有解决“异常结果是否足够聚焦”的问题，所以我们加入 Z-score 进行二次确认。

第 7 页：V2 IQR + Z-score 算法介绍

IQR 关注的是一个点是否超出分位数边界；Z-score 关注的是这个点相对均值偏离了多少个标准差。

Z-score 的公式是：

```
z = (value - mean) / std
```

它表示当前 value 距离该序列均值有多少个标准差。如果绝对值超过 3，一般认为它是显著偏离点。

从数据处理角度看，V2 仍然使用和 V1 一样的分组粒度：

```
object_id + kpi_name + metric_type
```

但是在每组里面，除了计算 IQR 的 Q1、Q3、上下界，还额外计算 `mean` 和 `std`。之后 join 回每个时间点，分别判断 IQR 异常和 Z-score 异常。

V2 最终采用严格策略：

```
is_anomaly_final = is_anomaly_iqr AND is_anomaly_zscore
```

也就是说，一个点必须同时被 IQR 和 Z-score 判为异常，才会被保留下来。

这样处理的原因是：V1 只使用单一规则，可能包含一些边界波动点；V2 用两个统计视角做交集，得到的是更高置信度的异常集合。

第 8 页：V2 IQR + Z-score 结果分析

V1 检测出 574,873 个异常点，而 V2 只保留 129,228 个。异常率从约 1.28% 降到约 0.287%。

这说明 V2 过滤掉了大量只被 IQR 单一规则判为异常、但没有明显偏离均值的点。因此 V2 的结果更加聚焦，噪声更少，也更符合“提高异常判断可靠性”的目标。

如果看不同类型，service 的收紧最明显，V2 只保留了 V1 中约 6.76% 的异常点；istio 的保留比例最高，说明 istio 中有更多 IQR 异常同时也明显偏离均值。

第 9 页：V2 结果分析：引出 V3

图中横轴是时间，纵轴是某个时间点附近的异常数量，不同颜色表示不同 `metric_type`。它展示的是 V2 检测出的高置信异常在时间上的分布。

从图中可以看到，异常不是均匀分布的，而是在某些时间段明显集中。例如 istio 在某些时间段出现了异常高峰，container 也存在多个波动段。

这说明监控数据存在明显的时间动态和局部波动。

但是 V2 的统计方式仍然是全局静态的。也就是说，对同一条指标序列，V2 会用整条序列的 mean 和 std 来判断每个点是否异常。这样可能会忽略一些局部时间段内的突刺。

所以我们后面做的一个方向就是从时间维度做滑动窗口。

第 10 页：V2 结果分析：引出 V4

图中展示的是 KPI 共现次数 Top 20。它不是看单个 KPI 的异常次数，而是看哪些 KPI 经常在同一对象、同一时间共同异常。

如果某些 KPI 组合的共现次数很高，说明异常并不是随机分散在单个指标上，而是存在指标之间的关联关系。比如容器网络接收、发送相关指标，Istio 内存相关指标，系统 IO 和网络指标，都可能一起出现异常。

这说明单指标检测还有一个不足：它只能判断某一个 `kpi_name` 的 value 是否异常，却不能表达“同一个对象同一时间多个指标共同异常”的情况。

所以我们后面做的另一个方向就是从指标维度做多指标融合。

第 11 页：V3 Window Z-score 算法介绍

V3 是滑动窗口 Z-score。

V2 的“全局”是指一整条指标序列共用一套 mean 和 std；V3 的“局部”是指每个时间点只参考它附近的一段窗口数据。

在我们实现中，窗口定义为：

```
当前点 + 前 100 个历史点
```

对窗口内的数据计算：

```
window_mean
window_std
window_z_score = (value - window_mean) / window_std
```

如果：

```
abs(window_z_score) > 3.0
```

并且窗口点数大于等于 30，就判为局部异常。

这里有两个参数需要解释。`3.0` 是 Z-score 异常检测常用的经验阈值，表示当前点偏离窗口均值超过 3 个标准差，属于显著偏离。窗口点数大于等于 30，是为了保证窗口均值和标准差相对稳定。如果窗口太短，统计量不可靠，容易造成误判。

V3 的核心变化不是换公式，而是改变统计范围：从整条序列的全局统计，变成当前时间附近的局部统计。

第 12 页：V3 Window Z-score 结果分析

V3 检测出 224,512 个异常点，异常率约 0.499%，约为 V2 的 1.74 倍。

这个结果不能简单理解为“V3 一定比 V2 更好”。更准确地说，V2 和 V3 关注点不同。V2 更保守，适合高置信异常；V3 更敏感，适合补充发现局部时间异常。

为什么 V3 会发现更多异常？因为有些点在整条序列的全局范围内不一定极端，但在当前局部时间段内明显偏离。例如某个服务晚上请求量平时很低，如果突然升高，放在全天数据里可能不算特别大；但放在晚上这一段局部窗口里，就是明显异常。

```
V3 使用局部窗口统计，能发现 V2 全局静态统计可能忽略的局部时间异常。
```

第 13 页：V3 Window Z-score 结果分析：service Top 序列

这一页进一步看 V3 在 service 类型中具体发现了什么。

图中展示的是 V3 service Top 异常序列。每一项表示某个服务对象的某个 KPI 时间序列中，有多少个时间点在局部窗口内被判为异常。

比如 `frontend-http / mrt` 有 255 个异常点，表示 frontend-http 的平均响应时间序列中，有 255 个时间点相对各自局部窗口出现明显偏离。

这张图有一个重要现象：V3 中不仅 `mrt` 响应时间异常明显，`count` 请求量指标也大量出现。

和 V2 对比，V2 的 service 异常主要集中在 `mrt`，`count` 只零星出现；而 V3 中 `count` 成为 Top 异常指标之一。这说明滑动窗口对局部请求量波动更敏感。

理论解释是：count 请求量通常具有明显的时间段变化，比如高峰期和低谷期。V2 使用全局 mean/std，可能会被全天不同阶段的流量差异拉宽正常范围；V3 只和当前窗口附近的正常水平比较，因此更容易发现局部突增或突降。

第 14 页：V4 MultiMetric Fusion 算法介绍

V4 是多指标融合异常检测。

前面 V1、V2、V3 都是单指标检测，也就是每次判断某个 `object_id + kpi_name + metric_type` 的 value 是否异常。

但真实系统故障往往不是一个指标孤立变化，而是同一个对象在同一时间多个指标一起异常。因此 V4 要判断的是多指标共同异常。

第一步是多指标对齐。我们按 `timestamp + object_id + metric_type` 分组，把多个 `kpi_name` pivot 到同一行。原来长表是一行一个 KPI，pivot 后变成一行包含多个 KPI 列。

第二步是单指标标准化。对每个 KPI 单独计算 Z-score：

```
z_i = (x_i - mean_i) / std_i
```

这里的 i 表示第 i 个 KPI。每个 KPI 都用自己的均值和标准差，因为不同 KPI 量纲不同，不能混在一起计算。

第三步是共同异常计数：

```
anomaly_metric_count = count(abs(z_i) > 3.0)
```

它表示同一对象同一时间有多少个 KPI 同时显著偏离。

如果：

```
anomaly_metric_count >= 2
```

就判为多指标融合异常事件。

最后 `max_abs_z_score` 表示这一行中偏离程度最大的 KPI，用来描述该融合事件的异常强度。

第 15 页：V4 MultiMetric Fusion 异常样例

这一页展示 V4 的具体异常样例。

以 service 类型为例，V4 选取参与融合的 KPI 包括 `count`、`mrt`、`sr` 和 `rr`。它们分别表示请求数量、平均响应时间、成功率和响应率。

表格中的每一行不是单个 KPI 的异常点，而是一个多指标融合异常事件。它的粒度是：

```
timestamp + object_id + metric_type
```

比如某一行中 `checkoutservice-grpc` 在某个时间点的 `anomaly_metric_count = 3`，表示同一时间这个服务有 3 个 KPI 同时超过异常阈值。

`max_abs_z_score` 表示这些异常 KPI 里偏离程度最大的那个值，用来表示异常强度。

这页要强调：V4 的输出不是“某一个 kpi_name 异常”，而是“某个对象在某个时间点多个 KPI 同时异常”。这比单指标异常更接近系统级异常事件。

第 16 页：V4 MultiMetric Fusion 结果分析

这一页展示 V4 的整体分布。

V4 总共检测出 73,958 个多指标融合异常事件。注意这里不能叫单指标异常点，因为 V4 的统计粒度和 V1、V2、V3 不同。

V1、V2、V3 的异常点粒度是：

```
timestamp + object_id + kpi_name + metric_type
```

而 V4 的融合异常事件粒度是：

```
timestamp + object_id + metric_type
```

也就是说，如果同一时间同一对象有 3 个 KPI 同时异常，V1/V2/V3 可能统计为 3 个单指标异常点；V4 会统计为 1 个融合异常事件。

从图中可以看到，多指标融合异常主要集中在 istio 和 container。istio 有 44,784 个，container 有 27,172 个。service 和 jvm 数量很少，分别只有 106 和 153。

这说明服务网格和容器层更容易出现多个 KPI 同时异常的系统性波动；而 service 和 jvm 的异常更多表现为单指标异常。

V4 的优势是：它不是简单保留所有异常点，而是进一步聚焦多指标共同异常，更接近系统级故障表现。

第 17 页：四个方案总结：能力边界与优化动机

这一页是四个版本的总结。

可以按照“每个版本解决什么问题”来讲。

V1 解决的是：在没有可靠人工标签的情况下，能不能建立一个简单、可解释的无监督 baseline。它的优势是易解释，适合大规模 Spark 统计；不足是比较敏感，异常点较多。

V2 解决的是：如何提高异常置信度。它用 IQR 和 Z-score 的交集过滤掉低置信异常，使结果更聚焦；不足是仍然使用全局静态统计，并且仍是单指标检测。

V3 解决的是：时间动态问题。它用滑动窗口让均值和标准差随局部时间变化，更容易发现局部突刺；不足是更敏感，可能带来更多候选异常。

V4 解决的是：指标关联问题。它把多个 KPI 放到同一行，判断是否多个指标同时异常，更接近系统级异常；但它和前三版统计粒度不同，不能简单当成单指标异常点比较。

这一页的核心结论是：

> 四个版本不是简单替代关系，而是从不同角度扩展异常检测能力。

第 18 页：四个方案总结：能力边界与优化动机

这一页可以结合前端或者图 13 继续总结。

如果页面上放的是四版本对比图，可以这样讲：

V1 作为 baseline 异常数量最大，说明它足够敏感；V2 明显收紧，说明双规则交集提高了异常置信度；V3 数量回升，说明局部窗口发现了更多时间局部异常；V4 数量较少，但它统计的是融合异常事件，说明它更关注多指标共同变化。

如果页面上放的是同一时间序列四版本异常标记对比，可以这样讲：

这张图把同一条时间序列分别用 V1、V2、V3、V4 标出异常位置。它能直观看到不同算法关注点不同：V1 标出的点较多，V2 更保守，V3 更关注局部时间窗口，V4 标记的是多指标融合事件发生的时间点。

这一页建议用一句话收束：

> V1 解决能不能做，V2 解决准不准，V3 解决是否考虑时间变化，V4 解决是否考虑多指标共同异常。

第 19 页：小组分工

这一页介绍小组分工。

马蕊主要负责 V3 时间动态异常检测与实验对比。她提出并验证了滑动窗口 Z-score 思路，用来解决 V1/V2 全局静态统计忽略局部时间变化的问题，并整理了 V2/V3 在异常数量、异常序列和 service Top 异常上的差异。

张玳瑗主要负责实验结果统计与可视化数据构建。她负责统一 V1–V4 的结果统计口径，整理 `anomaly_points`、`stats`、`summary` 等输出数据，提取总异常数、按 `metric_type` 分布、Top 异常序列、四版本对比表和时间序列展示数据，并协助构建前端 Dashboard 所需的 CSV/JSON。

李姝漫主要负责多指标融合与结果展示分析。她提出 V4 从单指标检测扩展到多指标融合检测的思路，设计 `timestamp + object_id + metric_type` 粒度下的多 KPI 共同异常判断，完成 V4 结果分析，并将结果转化为图表和前端展示逻辑。

王咏心主要负责数据理解与清洗建模。她分析老师提供的 6 个场景、5 类指标数据结构，提出统一成长表结构的方案，完成 service 宽表转长表、类型转换、空值过滤和 `clean_metrics` 构造方案，并协助验证 V1–V4 数据来源正确。

吴柯蝶负责项目统筹与核心算法实现。她提出四阶段迭代式异常检测路线，负责 V1/V2 核心实现、结果整合和答辩主线设计。同时负责 Hadoop、HDFS、Spark 环境配置，完成原始数据上传、路径验证、Spark 读取测试，并协助解决运行慢、内存压力和参数配置问题。

这里可以强调：每位成员都参与了数据、算法、实验或结果分析中的核心环节，而不是单纯边缘工作。

第 20 页：附录：V2 service Top 序列

这一页是附录页，主要用于老师追问时补充说明 V2 的 service 结果。

如果需要讲，可以这样说：

这是 V2 在 service 类型下的 Top 异常序列。可以看到 V2 的 service 异常主要集中在 `mrt`，也就是平均响应时间。例如 `currencyservice-grpc / mrt`、`frontend-http / mrt`、`checkoutservice-grpc / mrt` 等都比较突出。

这说明 V2 的高置信异常在 service 层更多表现为响应时间异常，而 `count` 请求量指标在 V2 中只零星出现。这也为后续 V3 的结果提供了对比基础：V3 中 `count` 指标大量出现，说明滑动窗口更容易捕捉局部请求量波动。

如果时间不足，这页可以不主动讲，只作为备用材料。

---

15 分钟压缩版讲法

如果只有 15 分钟，建议压缩如下：

```
第 1 页：10 秒，开场。
第 2 页：1 分钟，讲总览。
第 3–4 页：2 分钟，讲原始数据和 Spark 清洗。
第 5–6 页：2 分钟，讲 V1。
第 7–10 页：3 分钟，讲 V2 及引出 V3/V4。
第 11–13 页：2.5 分钟，讲 V3。
第 14–16 页：2.5 分钟，讲 V4。
第 17–18 页：1 分钟，总结四版本。
第 19 页：1 分钟，分工。
第 20 页：不主动讲，作为附录。
```

30 分钟展开版讲法

如果时间接近 30 分钟，可以展开：

```
数据部分重点讲原始格式差异、service 宽表转长表、clean_metrics 的意义；
V1 部分重点讲为什么分组；
V2 部分重点讲异常置信度和两个不足；
V3 部分重点讲全局与局部的区别；
V4 部分重点讲单指标异常点和融合异常事件的区别；
可视化部分可以现场切前端，展示 V1/V2/V3/V4 和图 13；
最后讲工程问题：环境配置、运行慢、内存压力、减少 action 和分批处理。
```

结尾总结

最后可以这样收尾：

```
本项目从原始 AIOps 监控 CSV 出发，先通过 HDFS 和 Spark 完成大规模数据存储与清洗，构造统一的 clean_metrics 时间序列数据。在此基础上，我们逐步实现 V1 到 V4 四个版本：从 IQR baseline，到 IQR + Z-score 高置信筛选，再到滑动窗口时间动态建模，最后扩展到多指标融合异常事件检测。

这次项目让我们体会到，大数据平台项目不只是写出算法，更重要的是让算法能够在真实规模的数据上稳定运行，并通过清晰的结果统计和可视化展示支撑分析结论。
```